



Documentation Technique : Suite de Tests Docker

Cette documentation décrit le fonctionnement, l'architecture et l'utilisation de la suite de validation intégrée au projet. Elle permet de garantir que les services (MariaDB, MQTT, Web, Grafana) communiquent correctement entre eux.

1. Architecture du Test Runner

Le service `test_runner` est un conteneur éphémère basé sur **Alpine Linux**. Il est conçu pour s'exécuter, valider l'infrastructure, puis s'arrêter.

Configuration Docker Compose

- **Image** : alpine
- **Dépendances** : Il ne démarre qu'après mariadb, mosquitto, phpmyadmin, grafana et web_app.
- **Outils installés au runtime** : curl, netcat-openbsd (nc), iputils (ping).
- **Volume** : Les scripts locaux du dossier `./tests` sont montés dans `/tests` à l'intérieur du conteneur.

2. Hiérarchie des Scripts de Test

La suite est organisée en trois niveaux de profondeur :

Niveau 1 : Validation de base (`basic_connectivity_tests.sh`)

C'est le point d'entrée principal. Il orchestre quatre tests unitaires :

1. **Base de données** : Vérifie le port TCP 3306 de MariaDB.
2. **MQTT** : Vérifie le port TCP 1883 de Mosquitto.
3. **Réseau** : Teste la résolution DNS interne et le ping entre services.
4. **Services Web** : Utilise `wget --spider` pour vérifier la réponse HTTP des interfaces (Port 80, 3000, 5000).

Niveau 2 : Tests Croisés (`container_connectivity_tests.sh`)

Ce script va plus loin en exécutant des commandes **à l'intérieur** des autres conteneurs actifs :

- **Depuis web_app** : Vérifie s'il voit la BDD et le Broker MQTT.
- **Depuis mariadb** : Teste la connectivité inverse et valide les requêtes SQL (création de table test, vérification des accès root).
- **Réseau Avancé** : Analyse la résolution DNS complète et affiche les statistiques de consommation (CPU/RAM) des conteneurs.

Niveau 3 : Validation Totale (full_infrastructure_validation.sh)

Script "maître" qui enchaîne les tests de Niveau 1 et de Niveau 2 pour une certification complète de l'infrastructure avant une mise en production.

3. Guide d'utilisation (Commandes)

Lancer la suite de base (Recommandé)

Bash

```
docker compose run --rm test_runner
```

Lancer un test spécifique (Exemple : MariaDB uniquement)

Bash

```
docker compose run --rm test_runner sh /tests/test_database.sh
```

Lancer la validation complète (Mode Expert)

Bash

```
docker compose run --rm test_runner sh /tests/full_infrastructure_validation.sh
```

4. Détails des Tests Unitaires

Script	Cible	Commande Clé	Objectif
test_database.sh	mariadb:3306	nc -z	Vérifier que le moteur SQL accepte les connexions.
test_mqtt.sh	mosquitto:1883	nc -z	Vérifier que le broker est prêt pour les capteurs.
test_network.sh	DNS Interne	ping -c 2	Valider que le réseau marais_net est fonctionnel.
test_ports.sh	Web/API	wget --spider	Vérifier que les serveurs web ne retournent pas d'erreur 404/500.

5. Diagnostic des Échecs

En cas de message **"FAIL"** :

1. **Vérifiez les logs du service concerné** : `docker compose logs [service]`.
2. **Vérifiez les permissions** : Assurez-vous que les scripts dans `./tests` sont exécutables (`chmod +x tests/*.sh`).
3. **Nettoyage** : Si le réseau semble corrompu, tentez un `docker system prune` suivi d'un redémarrage.